Implementation of a Real-Time Spectrum Analyzer: System Architecture, Block-Diagram Design, and a Review of SDR-Based Implementations

FPT University, Ho Chi Minh City, Vietnam

Abstract. Spectral analysis is a core task in communications, radar, industrial monitoring, power-quality measurement and cognitive-radio systems. General-purpose, bench-top spectrum analyzers give excellent performance but are costly and are usually not designed for embedded, real-time operation. This report summarizes the architecture of a real-time spectrum analyzer, combining (i) a review of an FPGA-based implementation that estimates a modified periodogram of the input signal and automatically detects and estimates the power and frequency of up to NS narrow-band signals for a user-defined analysis bandwidth, instantaneous dynamic range (IDR) and frequency resolution, (ii) a functional block-diagram design of a classical swept-tuned spectrum analyzer, covering the RF front end, tuning/down-conversion, detection, video filtering, display, and control-and-processing subsystems, (iii) a review of two software-defined-radio (SDR) based spectrum analyzer implementations reported in the literature, and (iv) a MATLAB-based simulation of the modified-periodogram front end and hardware peak detector, illustrating quantization, windowing and real-time display behaviour with the results obtained.

Keywords: spectrum analyzer, block diagram, FPGA, FFT, periodogram, software-defined radio, real-time signal processing, simulation.

Table 1. Members and task assignment.

STT	Name	Roll Number	Task
1	Nguyễn Chí Dũng	SE205066	25%
2	Phạm Đỗ Nguyên Giáp	SE204241	25%
3	Nguyễn Quốc Toàn	SE205091	25%
4	Trần Nhật Hào	SE204728	25%

1 Introduction

Spectral analysis — measuring how the power of a signal is distributed over frequency — underlies a wide range of disciplines, including communications, radar, oceanography, geophysics, industrial condition monitoring and electric-power-quality measurement. Detecting the number of signals present and estimating their parameters

is generally difficult in the time domain but becomes tractable once the signal is viewed in the frequency domain.

In industrial monitoring, for example, the vibration spectrum of a rotating machine can reveal incipient mechanical faults; in electric power systems, spectral analysis is used to quantify voltage distortion, flicker and unbalance. In defense and civilian communications, detecting and characterizing multiple simultaneous signals is central to electronic-warfare, radar and cognitive-radio applications, where a device must sense whether a frequency band is occupied before transmitting.

Classical swept-tuned spectrum analyzers sweep a narrow-band filter across the band of interest and display the resulting envelope. General-purpose bench instruments of this kind are accurate and flexible but are expensive and not well matched to embedded, real-time monitoring tasks. A lower-cost alternative is to digitize the signal and estimate its spectrum numerically, most commonly via the Discrete Fourier Transform (DFT), computed efficiently with a Fast Fourier Transform (FFT). Because uniform windowing in a DFT-based periodogram causes spectral leakage that limits the achievable dynamic range, non-uniform (tapered) windows are used to trade frequency resolution for improved dynamic range.

Field-programmable gate arrays (FPGAs) provide an attractive implementation platform for this kind of processing: compared with an ASIC they are reconfigurable and far cheaper to develop, and compared with a DSP or general-purpose CPU/GPU they offer higher throughput and lower power for streaming, pipeline-friendly workloads such as FFT-based spectral estimation. An alternative, increasingly popular implementation route is software-defined radio (SDR), where a low-cost RF front end streams digitized samples to a host PC or an embedded FPGA for spectral processing in software or firmware; Section 7 reviews two representative SDR-based designs from the literature. The remainder of this report first summarizes, at a system level, how an FPGA-based real-time spectrum analyzer is specified and structured (Section 2), then presents the functional block diagram of a spectrum analyzer designed for this course project (Section 3), reviews the underlying detection algorithm, representative case studies and the FPGA implementation results reported in the reference design (Sections 4–6), surveys two SDR-based spectrum analyzer implementations from the literature (Section 7), and presents a MATLAB-based simulation of the modified-periodogram front end and peak detector (Section 8), before concluding in Section 9.

2 System Overview

A real-time spectrum analyzer of this type accepts a complex baseband signal, obtained from IQ demodulation and sampling at rate fs, and returns a compact description of the signals present rather than the full spectrum. The user specifies four parameters: the analysis bandwidth BWA (equivalent to the “span” of a conventional analyzer), the maximum number of signals to detect NS, the minimum frequency separation the analyzer must resolve Δf, and the instantaneous dynamic range IDR — the largest power difference between two signals that must still be separable. From

these parameters the internal algorithm parameters (decimation factor, window type, FFT length, and detection thresholds) are derived automatically.

The analyzer's output is a signal description word (SDW): the number of detected signals together with their estimated center frequencies and powers. Reducing a continuous stream of samples to this compact word is what allows the design to be embedded — downstream logic only has to consume a short summary rather than a full periodogram every sweep. The signal model underlying the detector assumes the input is a sum of R narrow-band tones plus white Gaussian noise, which is realistic whenever the individual signal bandwidths are smaller than the required frequency resolution.

3 Spectrum Analyzer Block Diagram

To relate the algorithmic description above to a physical instrument, Fig. 1 shows the overall functional block diagram of a classical swept/heterodyne spectrum analyzer, as developed for this project. The signal path runs left to right through six functional groups — the RF front end, the tuning and down-conversion stage, detection, video filtering and display — while a control-and-processing group supervises the sweep, timing and user interface. Figs. 2–7 expand each group individually, in the same order in which the signal (and the sweep control loop) traverses the instrument.

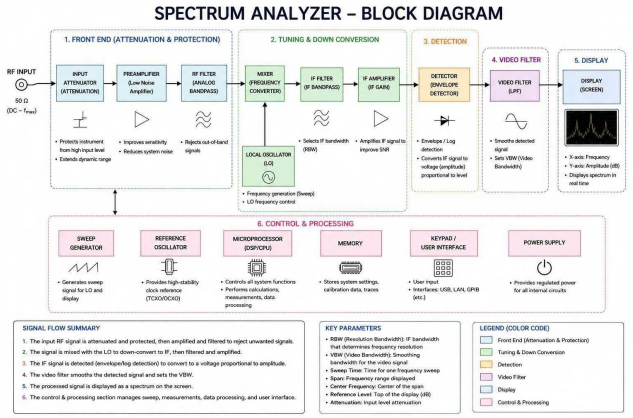

Fig. 1. Overall spectrum analyzer block diagram (signal path, groups 1–6).

3.1 Group 1 — Front End

The RF input first passes through a step attenuator, which protects the instrument from high input levels and extends its usable dynamic range. A low-noise preamplifier then improves sensitivity by reducing the effective noise contributed by later stages, after which an analog bandpass (RF) filter rejects out-of-band signals and images before the signal reaches the mixer.

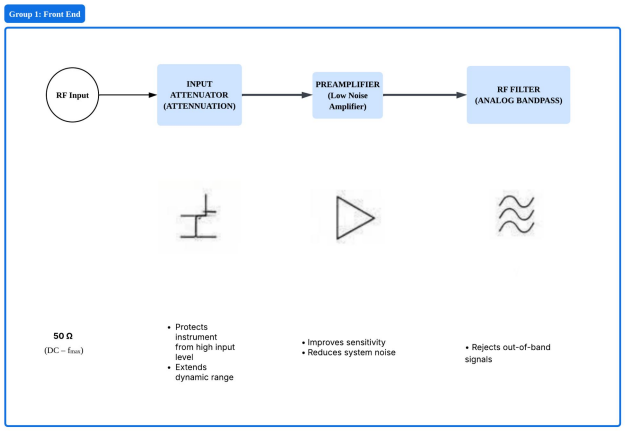

Fig. 2. Group 1 — Front end (attenuator, preamplifier, RF filter).

3.2 Group 2 — Tuning and Down-Conversion

A mixer, driven by a swept local oscillator (LO), translates the selected portion of the RF spectrum down to a fixed intermediate frequency (IF). The LO frequency is what is actually "swept" across the span. An IF bandpass filter selects the resolution bandwidth (RBW), and an IF amplifier restores gain and improves the signal-to-noise ratio (SNR) before detection.

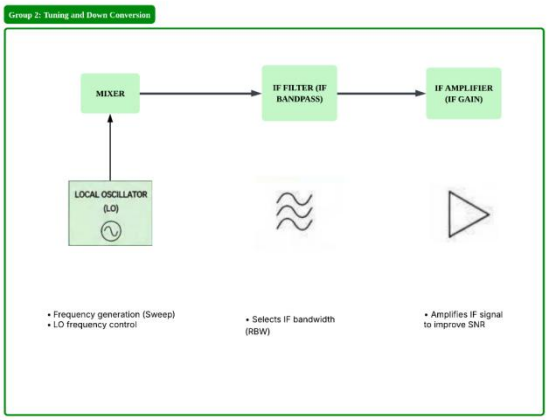

Fig. 3. Group 2 — Tuning and down-conversion (mixer, LO, IF filter, IF amplifier).

3.3 Group 3 — Detection

An envelope/log detector converts the filtered IF signal into a video (baseband) voltage proportional to its amplitude or its logarithm, so that the wide dynamic range

of typical RF signals can be compressed onto a linear display scale expressed in decibels.

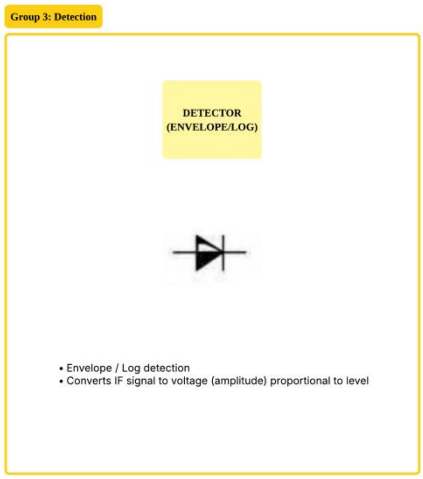

Fig. 4. Group 3 — Detection (envelope/log detector).

3.4 Group 4 — Video Filter

A low-pass video filter smooths the detected envelope, trading trace noise for sweep speed. Its cutoff — the video bandwidth (VBW) — is normally set relative to the RBW to control the visible noise floor without excessively slowing the sweep.

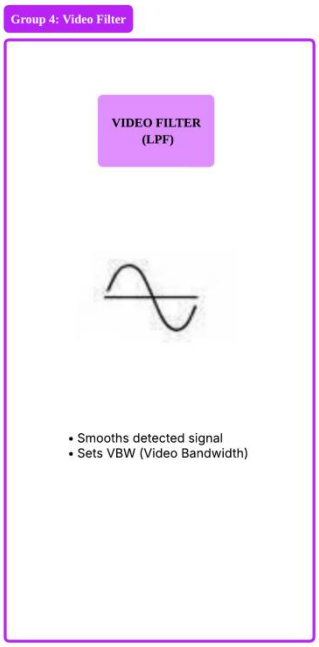

Fig. 5. Group 4 — Video filter (low-pass filter, VBW).

3.5 Group 5 — Display

The processed video signal is presented with frequency on the horizontal axis and amplitude, in decibels, on the vertical axis, refreshed in real time as the sweep proceeds, so the operator can see the instantaneous power spectrum of the input.

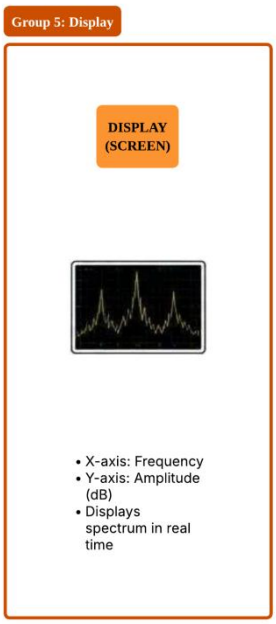

Fig. 6. Group 5 — Display (frequency vs. amplitude).

3.6 Group 6 — Control and Processing

A sweep generator drives the LO synchronously with the display timebase, while a high-stability reference oscillator (TCXO/OCXO) provides the timing backbone for the whole instrument. A central microprocessor (DSP/CPU) sequences the sweep, performs measurement calculations and coordinates the other blocks; memory stores settings, calibration data and captured traces; a keypad/user-interface block handles operator input and remote-control ports (USB, LAN, GPIB); and a power supply furnishes regulated rails to every subsystem. This group closes the loop back to the front end/LO, so the instrument continuously repeats the sweep–detect–display cycle.

Signal-flow summary: (1) the RF input is attenuated, amplified and filtered; (2) it is mixed with the LO and down-converted to IF, then filtered and amplified again; (3) the IF signal is envelope/log-detected into a video voltage; (4) the video filter smooths this voltage and sets the VBW; (5) the result is displayed as amplitude versus frequency; (6) the control-and-processing group drives the sweep, timing, measurements and user interface throughout.

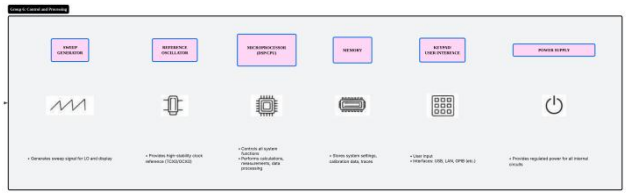

Fig. 7. Group 6 — Control and processing (sweep generator, reference oscillator, microprocessor, memory, keypad/UI, power supply).

4 Detection Algorithm Overview

Internally, the reference FPGA design implements the same signal path described above in the digital domain, in three stages. First, bandwidth adaptation filters the complex input with an anti-aliasing FIR filter and decimates it by a factor L so that only the requested analysis bandwidth BWA is processed, which minimizes the computational load of later stages. Second, a modified periodogram is formed by windowing the decimated samples, computing an NFFT-point FFT, and taking the squared magnitude of each bin; the window shape controls the trade-off between side-lobe level (which sets the achievable IDR) and main-lobe width (which sets the achievable frequency resolution). Third, a periodogram-analysis (PA) stage searches the spectrum for up to NS local maxima, estimates the noise floor from the remaining bins, applies a constant-false-alarm-rate threshold, and reports the frequency and power of every bin that survives both the noise threshold and a secondary threshold used to reject window side lobes.

Because a stronger window main lobe occupies several adjacent FFT bins, the algorithm removes the bins around each detected maximum before searching for the next one, which prevents a single strong tone from being reported multiple times and also determines the practical frequency resolution that can be achieved for a given number of simultaneous signals.

5 Representative Case Studies

The reference design was validated with three case studies of increasing difficulty. In the first, the analysis bandwidth equals the sampling frequency (no anti-aliasing filter is required) with a modest 25 dB dynamic-range requirement, met with a Hamming window. In the second, a tighter frequency resolution is required with the analysis bandwidth narrower than the sampling frequency, so a decimating FIR filter is introduced; a Hamming window still satisfies the 30 dB dynamic-range target. In the third, the dynamic-range requirement rises to 70 dB, which a Hamming window cannot satisfy; a Blackman–Harris window, with much lower side lobes, is used instead at the cost of a wider main lobe and therefore coarser resolution. In all three cases a 4096-point FFT was sufficient, and Monte-Carlo simulation showed correct

detection probability approaching unity once the signal-to-noise ratio exceeded roughly −20 dB to −25 dB, depending on the case.

6 FPGA Implementation

The hardware architecture mirrors the three algorithmic stages — bandwidth adaptation, modified periodogram, and periodogram analysis — each running in its own clock domain so that filtering, FFT computation and detection can be pipelined and clocked independently for the best performance–area trade-off. The FIR filtering and decimation stage is implemented with a polyphase structure that fits efficiently into dedicated FPGA DSP and memory resources. Because the design targets a large, 4096-point transform that must process a continuous sample stream while conserving FPGA area, a resource-saving (iterative) radix-4 FFT architecture was chosen over a fully pipelined one, accepting somewhat lower throughput in exchange for a much smaller silicon footprint. The periodogram-analysis block then searches the (unordered) FFT output for the strongest bins over several iterations, estimates the noise floor from the bins that remain, and applies the detection thresholds before a control module assembles the final signal description word. On a mid-range Xilinx Virtex-5 device, the complete design occupied under 10% of the FPGA's logic resources in every case study, while meeting the continuous-input, real-time throughput required by the specification.

7 Related SDR-Based Spectrum Analyzer Implementations

Besides the FPGA reference design reviewed above, two recent studies illustrate how a similar spectrum-analysis function can be realized with software-defined radio (SDR) front ends rather than a bespoke RF chain, and are summarized here for comparison with the block-diagram design of Section 3.

7.1 Low-Cost SDR Spectrum Analyzer for Digital Terrestrial Television Signals

Andrade et al. describe a prototype spectrum analyzer built around an inexpensive RTL-SDR dongle and GNU Radio to observe Digital Terrestrial Television (DTT) signals in the 470–698 MHz band used by the ISDB-Tb One-seg service in Quito, Ecuador [4]. Rather than a dedicated USRP-class device, the authors deliberately chose a low-cost RTL-SDR receiver so that the entire signal chain, aside from basic analog front-end components, is realized in software, with a low-pass filter used ahead of the analyzer to reduce out-of-band noise. The resulting tool was used to capture and visualize DTT multiplex signals from several locations around the city, functioning as a low-cost virtual laboratory instrument for telecommunications education as well as a means of checking regulatory compliance and signal quality; in the course of this testing, the authors reported that some received channels lacked

playable audiovisual content, indicating a transmission or reception problem worth further investigation. This design demonstrates that even a minimal, single-chip SDR receiver can reproduce the basic front-end/tuning/display function of a conventional spectrum analyzer (Groups 1–2 and 5 in Section 3) once the analog RF chain is replaced by a wideband ADC and the remaining processing is moved into software.

7.2 Hardware-Accelerated SDR Spectrum Analyzer with Fast Broadband Sweep

Flak presents a more elaborate SDR-based sensor built on a LimeSDR-USB board with custom FPGA firmware, aimed at broadband swept-FFT spectrum sensing for applications such as propagation studies, multiband occupancy monitoring and detection of unused ("white-space") spectrum [5]. Rather than streaming raw IQ samples to a host PC for all processing, as in a conventional SDR spectrum analyzer, the design moves the Welch spectral-density estimate — essentially an averaged, windowed periodogram similar in spirit to the modified periodogram of Section 4 — into the FPGA fabric, which both reduces the volume of data that must cross the USB link and offloads the host CPU. The frequency-tuning state machine and a calibration-value cache are likewise implemented in the FPGA so that the local oscillator can be retuned, and the resulting spectral snapshot accepted, with minimal dead time between steps of the sweep. The reported system achieves as much as 96 MHz of real-time instantaneous bandwidth and a cumulative sweep time of well under one millisecond per gigahertz of span, figures the author attributes largely to moving the spectral-estimation and tuning-control functions from host software into dedicated hardware. This mirrors, in a modern SDR context, the same motivation that underlies the FPGA reference design of Sections 4–6: pushing the periodogram computation and control logic as close to the ADC as possible to sustain a continuous, real-time data stream.

Taken together, the two SDR-based designs bracket the design space explored in this report from the opposite direction: instead of a swept analog front end (Section 3) driving a digital back end, they replace most of the analog signal chain with a wideband digitizer and move the equivalent of Groups 2–6 into software or FPGA firmware, at the cost of the tuning range and dynamic range that a purpose-built RF front end can provide. This reinforces the trade-off discussed in the report's conclusion: FPGA area and cost can be spent either on a fully digital back end behind a conventional swept front end, as reviewed in Sections 4–6, or on offloading an SDR's spectral estimation and tuning control, as in [4] and [5], with the right balance depending on the required bandwidth, dynamic range and unit cost.

8 Simulation Results

To complement the block-diagram design (Section 3) and the detection-algorithm review (Section 4), the modified-periodogram front end and hardware peak detector were reproduced in a MATLAB simulation. A synthetic input signal was generated,

passed through a model of a 12-bit ADC, windowed, transformed with an NFFT-point FFT, and processed by a simple local-maximum peak detector representative of the comparator/finite-state-machine implementation that would be used on an FPGA. The following figures summarize the resulting behaviour at each stage of this simulated pipeline.

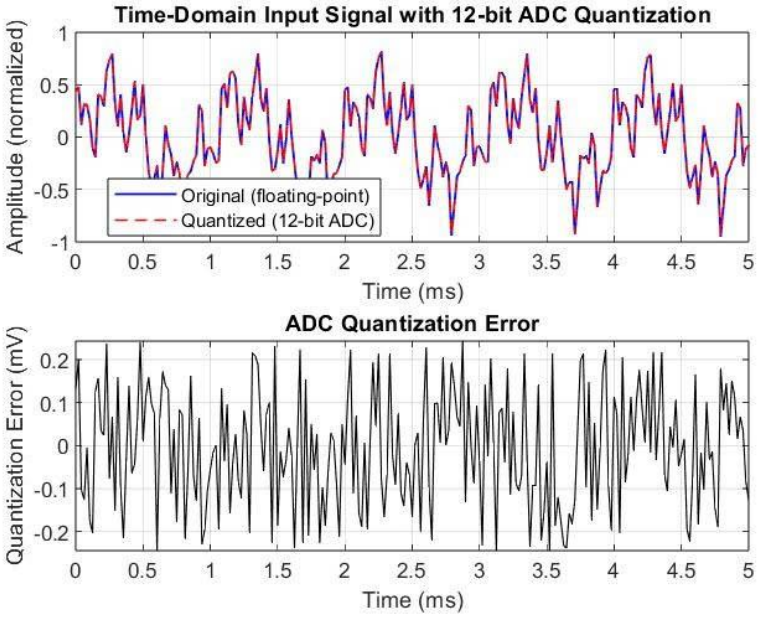

Fig. 8. Time-domain input signal and 12-bit ADC quantization error. The top plot shows the original floating-point signal (blue) and the quantized signal after 12-bit ADC simulation (red dashed); the bottom plot shows the corresponding quantization error.

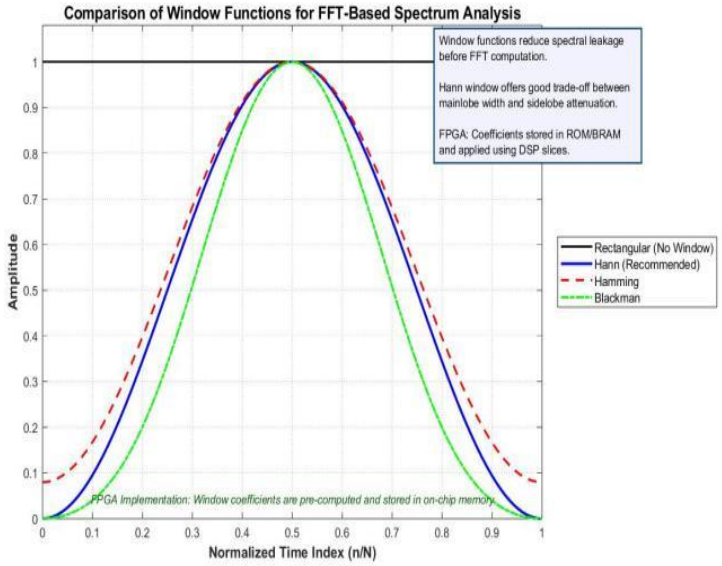

Fig. 9. Comparison of common window functions used prior to FFT computation. The Hann window is selected in this work for its favorable trade-off between main-lobe width and side-lobe attenuation; in the FPGA implementation, window coefficients are pre-stored in on-chip ROM/BRAM and applied using dedicated DSP slices.

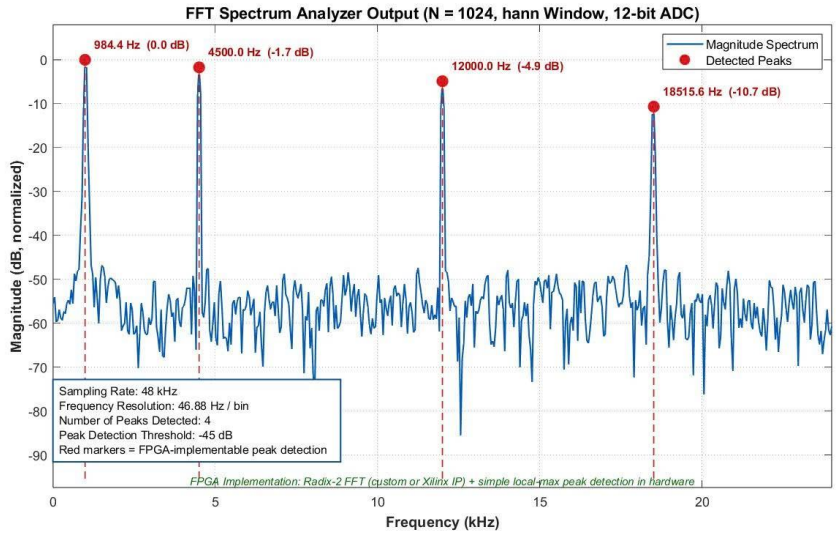

Fig. 10. Output spectrum of the proposed FPGA-based spectrum analyzer, using a 1024-point FFT with Hann windowing and 12-bit ADC quantization. Red markers indicate the peaks automatically detected by the hardware peak-detection module.

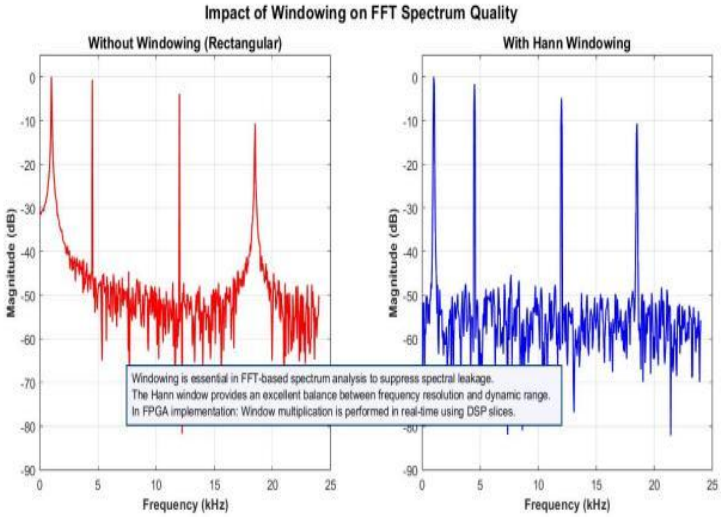

Fig. 11. Comparison of the FFT spectrum with and without windowing. Without windowing, severe spectral leakage is observed due to high side lobes; the Hann window significantly suppresses side lobes, resulting in cleaner spectral peaks. Windowing is performed in the FPGA using pre-stored coefficients in ROM/BRAM and real-time multiplication via DSP slices.

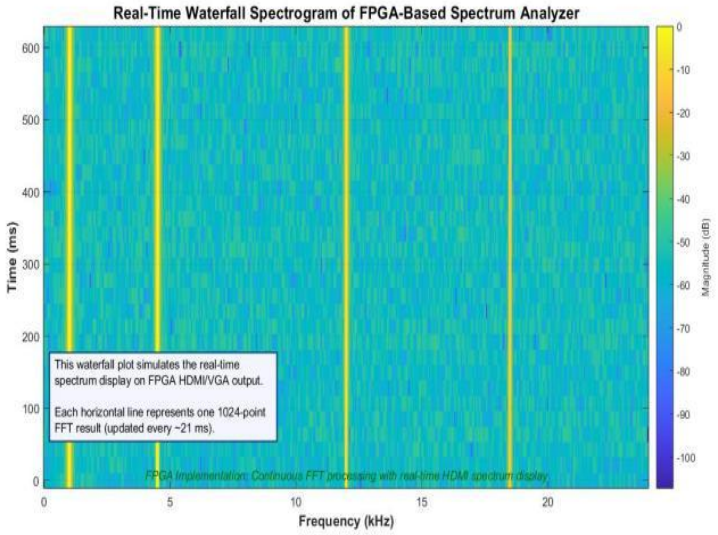

Fig. 12. Real-time waterfall spectrogram generated by the FPGA-based spectrum analyzer. Each horizontal slice corresponds to one 1024-point FFT result, updated approximately every 21 ms, demonstrating continuous real-time spectrum monitoring with HDMI display output.

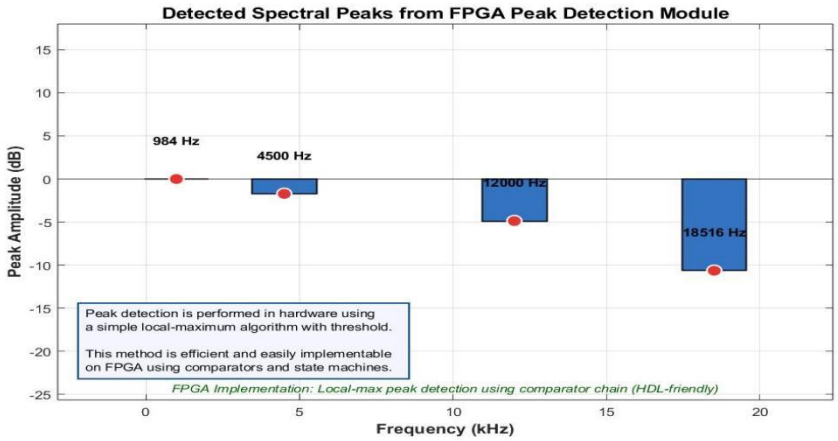

Fig. 13. Detected spectral peaks using the hardware-implemented peak-detection module. A simple local-maximum algorithm with a threshold is used, which is efficient and easily realizable on FPGA using comparators and finite-state machines.

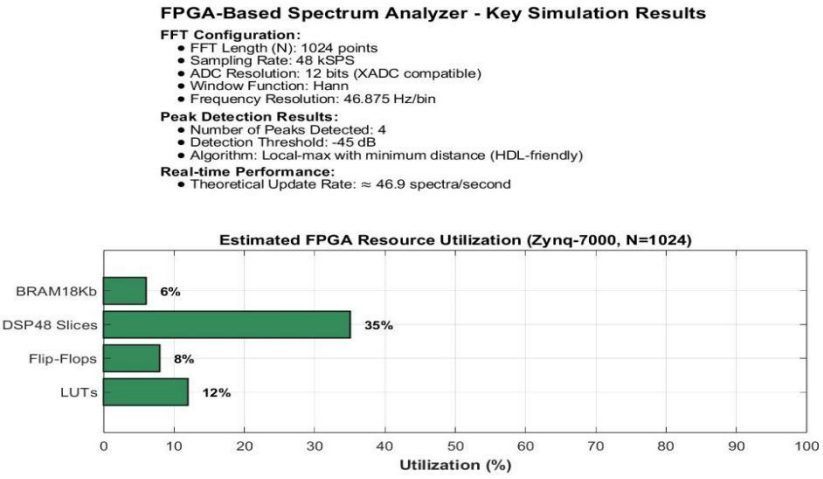

Fig. 14. Summary of key simulation results and estimated FPGA resource utilization on a Zynq-7000 device for N = 1024. The design achieves real-time spectrum analysis with moderate resource consumption, leaving room for additional features or multi-channel extensions.

Overall, the simulation results are consistent with the algorithmic description of Section 4: the Hann window used in Figs. 9 and 11 trades a wider main lobe for lower side lobes, which is what allows the peak detector of Figs. 10 and 13 to separate closely spaced tones from the noise floor and window artefacts; the waterfall display

of Fig. 12 illustrates the same continuous sweep–detect–display cycle described qualitatively for the analog instrument in Section 3; and the resource summary in Fig. 14 is in line with the low FPGA utilization reported for the reference hardware design in Section 6, confirming that the modified-periodogram approach remains inexpensive to implement even before hardware-specific optimization.

9 Conclusion

This report combined a system-level review of an FPGA-based real-time spectrum-analysis technique with a functional block-diagram design covering the classical front-end, tuning, detection, video-filter, display and control subsystems of a spectrum analyzer, a review of two SDR-based spectrum analyzer implementations from the literature, and a MATLAB-based simulation of the modified-periodogram front end and peak detector. Together they illustrate how the high-level signal path of a conventional swept analyzer (Section 3) corresponds to the digital processing stages — bandwidth adaptation, modified periodogram estimation, and periodogram analysis — used to obtain a compact, real-time description of the signals present in a band of interest (Sections 4–6 and 8).

Working through the block diagram made it clear why each stage exists and how the choices at one stage constrain the others. The front end (Group 1) sets the achievable dynamic range before any digital processing even begins: too little attenuation risks compressing the mixer, while too much attenuation buries weak signals in noise, so the attenuator and preamplifier settings must track the expected input level. The tuning and down-conversion stage (Group 2) shows why resolution bandwidth and sweep time are linked — a narrower RBW filter improves frequency resolution but takes longer to settle at each LO step, which is the direct hardware analogue of the FFT-length/dynamic-range trade-off examined in the window-selection discussion of Section 4. Detection and video filtering (Groups 3–4) then compress the wide dynamic range of RF signals onto a usable display scale, mirroring the periodogram's squared-magnitude and thresholding steps. Finally, the control-and-processing group (Group 6) ties the whole instrument together, closing the loop from sweep generation back to the front end in exactly the way the FPGA design's clock domains and control module coordinate the FIR, FFT and PA stages to sustain a continuous, real-time data flow.

From the algorithmic side, the case studies in Section 5 confirm that the same architecture can be retargeted to very different requirements simply by changing the window function, decimation factor and FFT length — a narrow-dynamic-range, wide-bandwidth application (Case 1) and a high-dynamic-range, narrow-bandwidth application (Case 3) are handled by the same processing chain, just with different parameters. The FPGA implementation results in Section 6 further show that this flexibility does not come at a large hardware cost: even the most demanding case occupied under 10% of the target device's resources, which suggests there is comfortable headroom to add further processing (for example, additional detection channels or a higher-resolution FFT) within the same platform. The SDR-based

designs reviewed in Section 7 show that this trade space extends beyond a fixed swept front end: replacing the analog RF chain with a wideband digitizer and moving detection and control into software or FPGA firmware, as in [4] and [5], can substantially lower cost and enable field or classroom deployment, at the expense of the tuning range and raw dynamic range that a purpose-built instrument provides. The simulation results in Section 8, in turn, confirm at a numerical level that the windowing and peak-detection choices discussed algorithmically in Section 4 behave as expected and remain inexpensive to implement, consistent with the low resource utilization reported for both the FPGA reference design and the SDR implementations.

Overall, the exercise of mapping a textbook block diagram onto an actual FFT-based implementation, supported by a numerical simulation and by two SDR-based designs from the literature, reinforced two points that are easy to miss when they are studied separately. First, a real-time spectrum analyzer is fundamentally a pipeline: every stage, whether analog, digital or software-defined, has to keep pace with a continuous input stream, and the system's overall latency and throughput are set by whichever stage is slowest, not by an average of the stages. Second, the four user-facing parameters — analysis bandwidth, number of signals, frequency resolution and dynamic range — are not independent; tightening one (for instance, demanding higher dynamic range) generally forces a compromise elsewhere (in this case, coarser frequency resolution, via the choice of window), a trade-off that recurs, in different guises, in both the FPGA reference design and the SDR-based literature reviewed here. Recognizing this trade-off is what ultimately connects the physical block diagram in Section 3 to the mathematical detection algorithm in Section 4 and to the simulation results in Section 8, and is the main takeaway of this project.

As a direction for future work, the group's block-diagram design could be extended with a digital IF/ADC stage between Groups 2 and 3, allowing the detection, video-filter and display blocks to be implemented digitally as in the reference FPGA design and the SDR-based sensors of Section 7; this would let a future iteration of the project prototype the periodogram-analysis algorithm directly against the block diagram developed here — and validate it against real captured data, as in [4] and [5] — rather than relying solely on the MATLAB simulation of Section 8.

References

1. V. Iglesias, J. Grajal, M. A. Sánchez, and M. López-Vallejo, "Implementation of a real-time spectrum analyzer on FPGA platforms," IEEE Trans. Instrum. Meas., vol. 64, no. 2, pp. 338–355, Feb. 2015.
2. F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," Proc. IEEE, vol. 66, no. 1, pp. 51–83, Jan. 1978.
3. M. T. Hunter, A. G. Kourtellis, C. D. Ziomek, and W. B. Mikhael, "Fundamentals of modern spectral analysis," IEEE Instrum. Meas. Mag., vol. 14, no. 4, pp. 12–16, Aug. 2011.

4. K. Andrade, F. Patiño, and T. Sánchez-Almeida, “A spectrum analyzer in the 470 to 698 MHz band using software defined radio for the analysis of digital terrestrial television signals (DTTs),” Eng. Proc., vol. 47, no. 1, art. 22, Dec. 2023.
5. P. Flak, “Hardware-accelerated real-time spectrum analyzer with a broadband fast sweep feature based on the cost-effective SDR platform,” IEEE Access, vol. 10, pp. 110934–110946, 2022.